

The Product Catalog and Shopping Cart.

This will introduce session management (remembering what a user did across different pages) and slightly more complex business logic, while still keeping the database out of the picture for now.

Here is your next assignment:

Mini Project 2: E-Commerce Product Catalog & Shopping Cart

Objective: Develop a module where users can view available products, add them to a virtual shopping cart, and view their total checkout amount using Spring MVC.

Architecture: UI → Controller → Service → Repository

Storage: In-memory collections (`ArrayList` , `HashMap` , etc.). No database connectivity yet.

Functional Requirements (FR)

1. Product Listing Dashboard

- **Action:** Display a list of all available products.
- **Output:** A page showing product names, prices, and available stock.
- *Note: Pre-populate your Repository with 4 or 5 dummy products when the application starts.*

2. Add to Cart Functionality

- **Fields:** Product ID, Quantity.
- **Validations (Backend):** * Check if the requested quantity is greater than 0.
 - Check if the requested quantity exceeds the available stock.
- **Output:** If successful, reduce the stock in the repository and redirect the user back to the product list with a "Successfully added to cart" message. If it fails, show an "Insufficient Stock" error.

3. View Shopping Cart

- **Action:** View all items currently in the user's cart.

- **Output:** Display a table showing the Product Name, Quantity, Subtotal (Price × Quantity), and a **Grand Total** at the bottom of the page.

Technical Requirements

Models:

- **Product** : id, name, price, stockQuantity
- **CartItem** : product, quantity, subtotal

Repository Layer:

- **ProductRepository** : Pre-loaded `ArrayList<Product>` . Methods to get all products, get a product by ID, and update stock.

Service Layer:

- **ProductService** : Fetch products.
- **CartService** : Handle the logic of calculating subtotals, grand totals, and ensuring stock isn't over-drafted.

Controller Layer (**ProductController** & **CartController**):

- Endpoints for listing products (`/products`).
- Endpoint for viewing the cart (`/cart`).
- Endpoint for adding to the cart (`/cart/add`).
- **New Concept Challenge:** Use `HttpSession` in your Controller to store the user's active cart. Since there is no database, the session will remember what they added while they click around the site.

Expected Screens:

1. **Product Catalog Page:** A table or list of items with an "Add to Cart" form/button next to each.
2. **Shopping Cart Page:** The summary of their order and the total price.

This project perfectly bridges the gap between simple data submission (like your Auth module) and actual business logic.

How does this scope look to you for your next milestone, or do you have a different e-commerce feature in mind you'd rather tackle first?